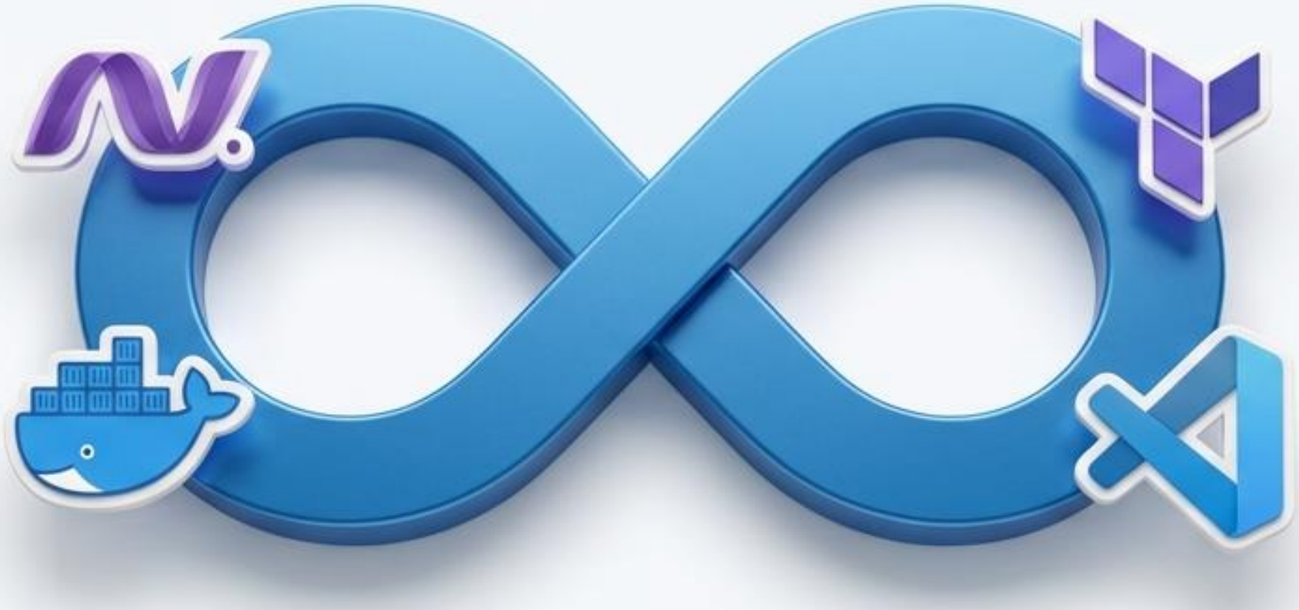
Dominando Azure DevOps: De la Idea al Despliegue

Una guía práctica basada en el flujo real de desarrollo de software.

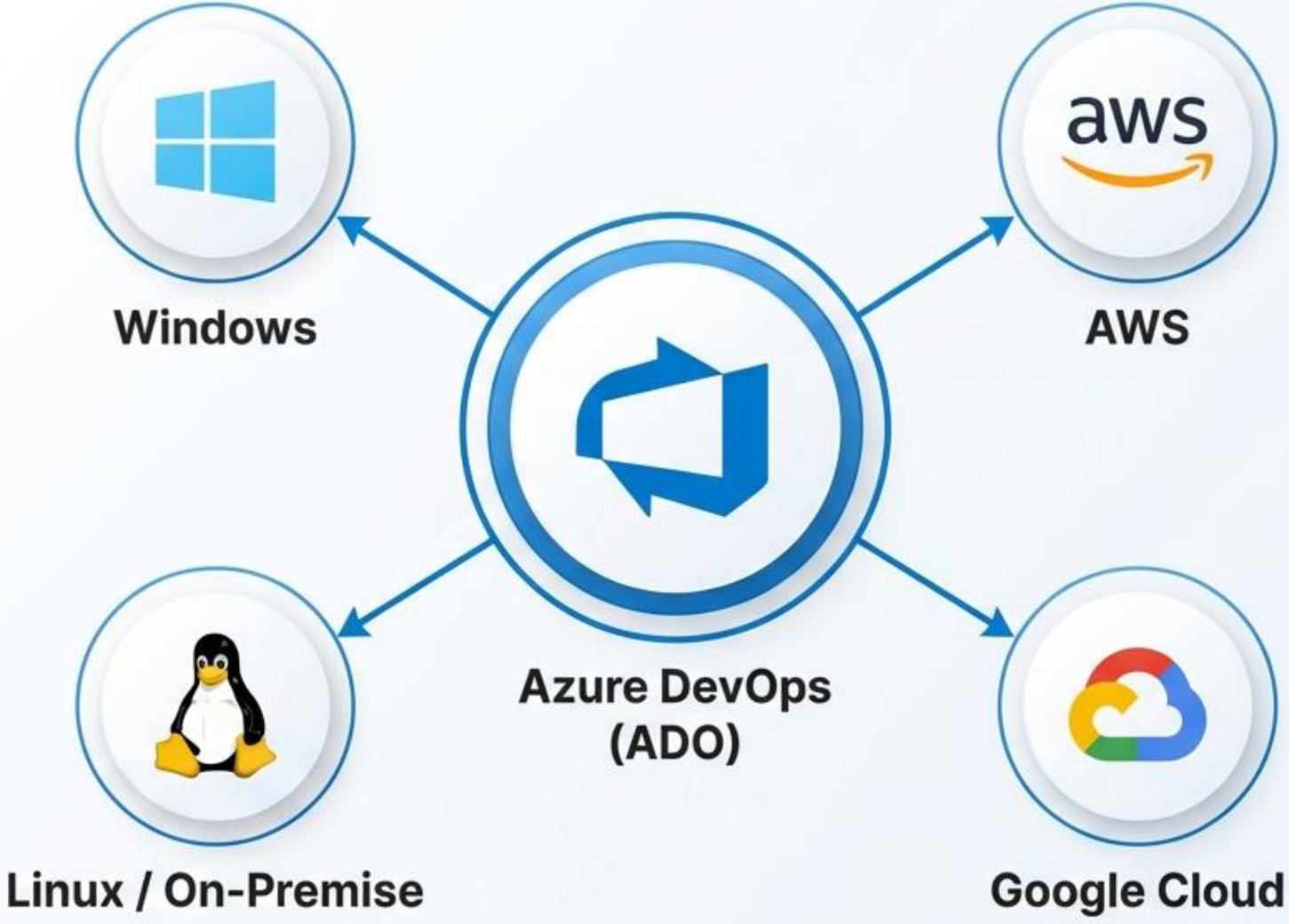


- Entender el ecosistema ADO y sus 5 pilares
- Caso de Estudio: API .NET + Docker + Azure Container Apps
- Flujo Real: Del Backlog al Código y la Nube

NotebookLM

¿Qué es Azure DevOps (ADO)?

- Suite SaaS para gestionar proyectos de software fin-a-fin
- **Agnóstico**: Funciona con Java, Python, Node, .NET
- **Multi-Nube**: Despliega donde quieras, no solo en Azure
- **Flexible**: Integra GitHub, Jenkins o herramientas externas



NotebookLM

La Plataforma: Services vs. Server

Azure DevOps Services

- Versión Nube (SaaS)
- Actualizaciones automáticas
- Escalable y mantenido por Microsoft
- Se organiza en "Organizaciones"

Azure DevOps Server

- Versión On-Premise (antes TFS)
- Tú gestionas el hardware
- Actualizaciones manuales
- Requiere mantenimiento de infraestructura

NotebookLM

El Ecosistema: Los 5 Pilares

Azure Boards	Planear y Gestionar (Agile/Scrum)
Azure Repos	Código Fuente y Git
Azure Pipelines	CI/CD y Automatización
Azure Test Plans	Pruebas y Calidad
Azure Artifacts	Paquetes y Librerías

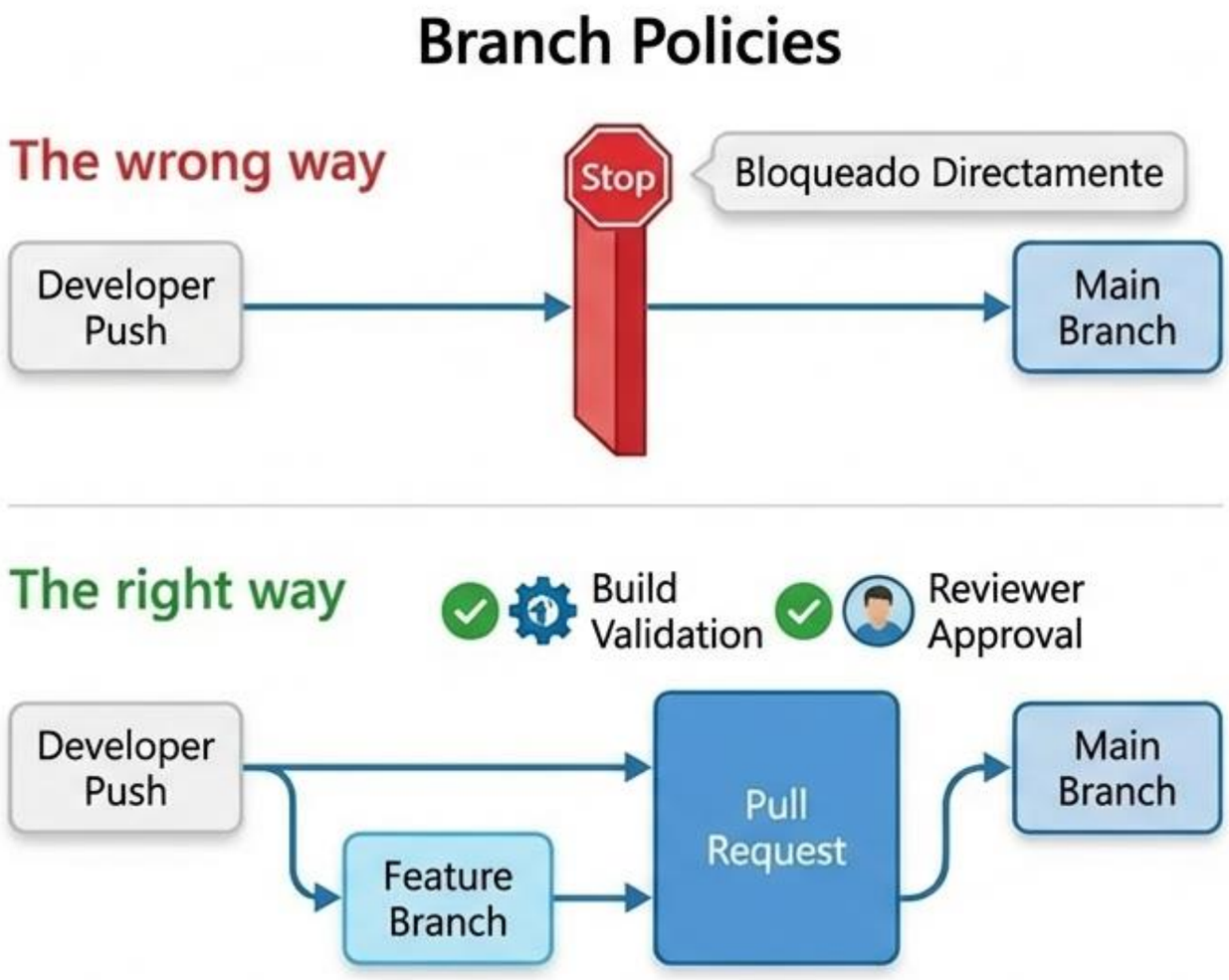
NotebookLM

Azure Boards: Donde nace el trabajo

- **Backlog:** Lista de requerimientos (User Stories)
- **Tareas:** Pasos técnicos específicos
- **Criterios de Aceptación:** Definición de ‘Terminado’
- **Asignación:** ¿Quién hace qué?
- **Estados:** Visualización del flujo de trabajo

NotebookLM

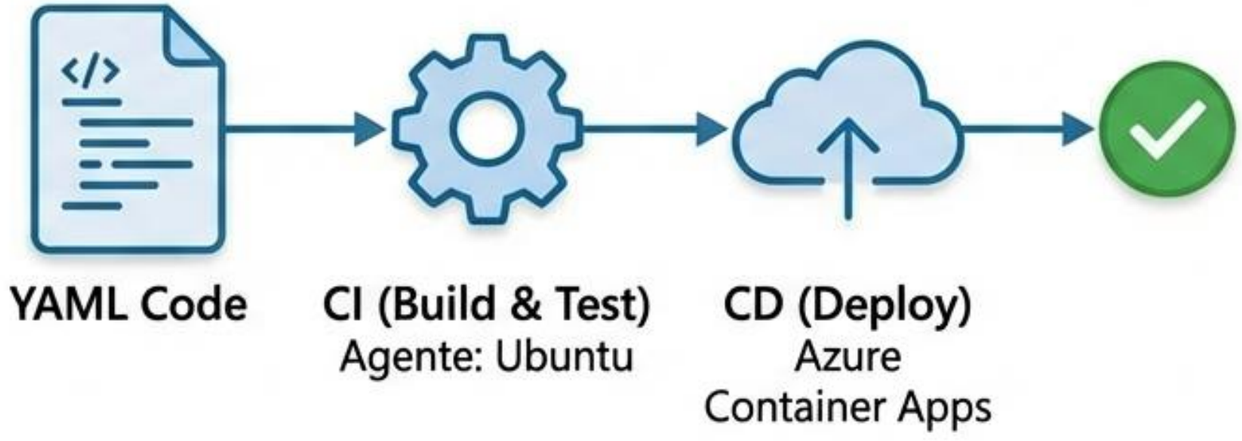
Azure Repos: Seguridad y Control



- **Branch Policies:** La regla de oro para proteger Main
- **Sin Push Directo:** Todo cambio requiere PR
- **Reviewers:** Mínimo 1 par de ojos humanos
- **Build Validation:** El código debe compilar antes de entrar

NotebookLM

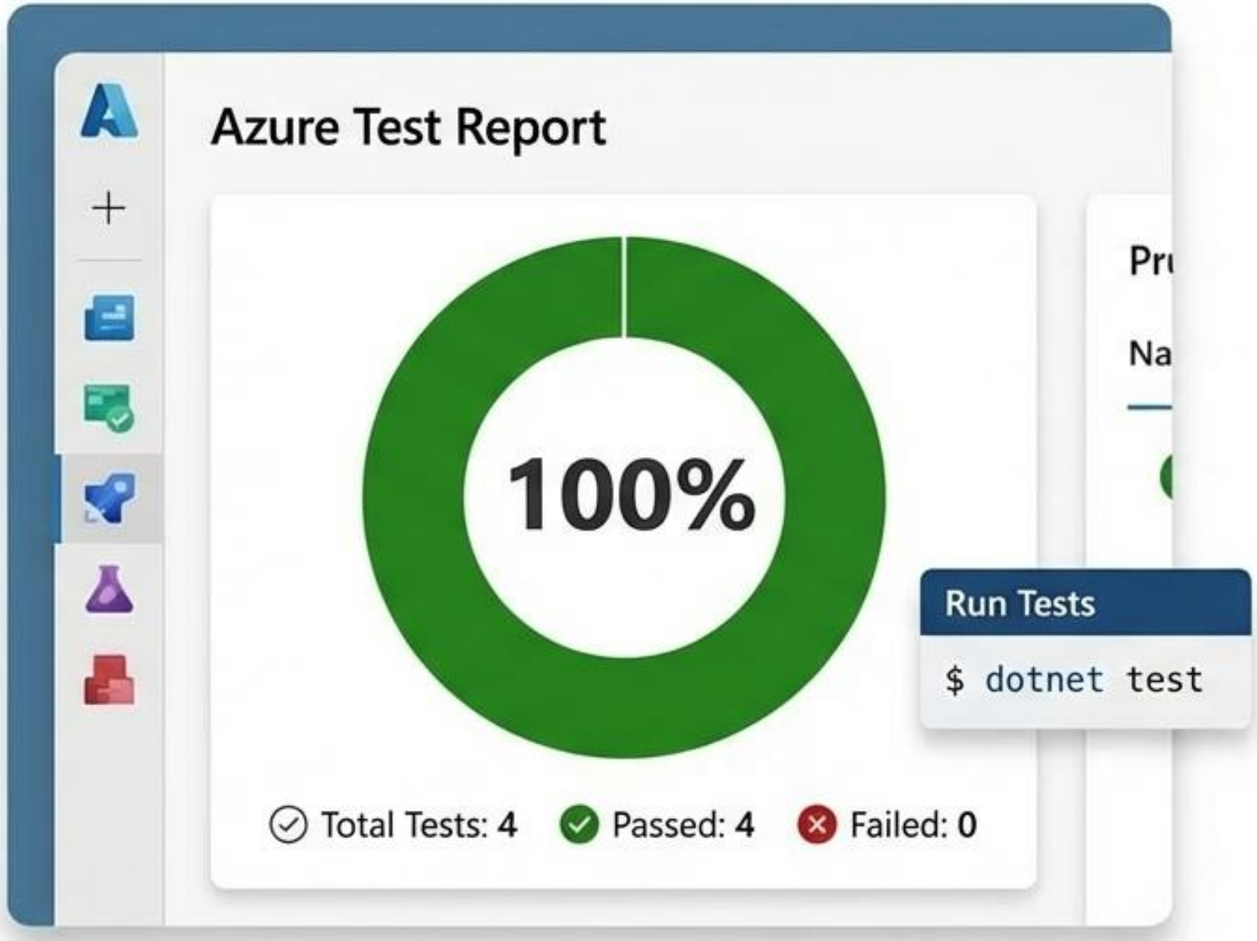
Azure Pipelines: Automatización (CI/CD)



- Definido en código (YAML)
- **CI (Integración Continua):** Compila y prueba en cada cambio
- **CD (Entrega Continua):** Despliega automáticamente
- **Agentes:** Máquinas virtuales que ejecutan el trabajo.

NotebookLM

Azure Test Plans y Reportes



- Visibilidad automática de resultados
- **Pruebas Unitarias** ejecutadas por el Pipeline
- **Code Coverage**: % de código analizado
- **Meta**: 100% Passed antes del Merge

NotebookLM

Azure Artifacts y Paquetes

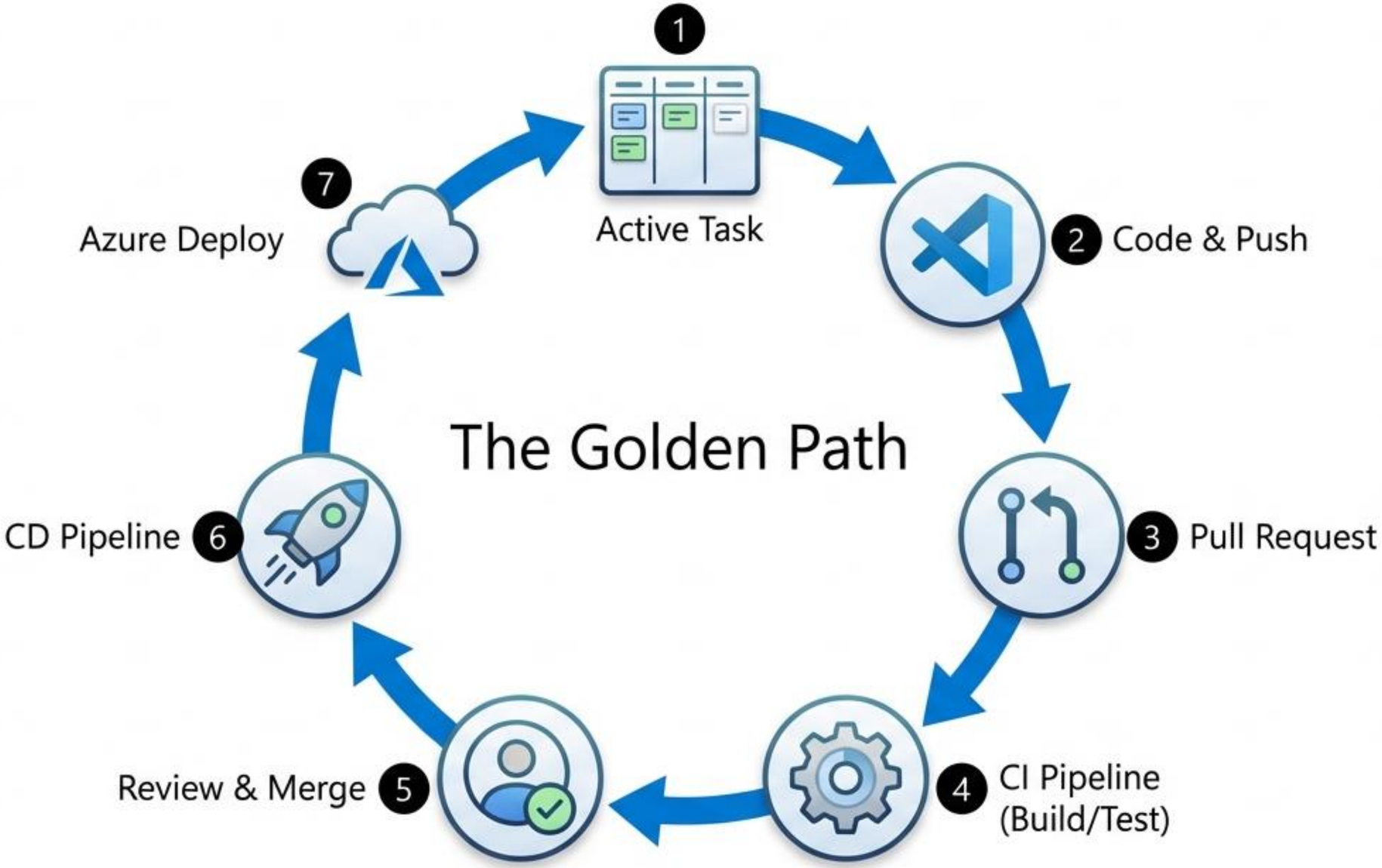
Librerías / Paquetes

Imágenes Docker

- Almacén de dependencias (NuGet, npm, Maven)
- Comparte código entre equipos internos
- **Pipeline Artifacts**: Mueve archivos entre etapas
- **Docker Registry**: El ‘artefacto’ para contenedores

NotebookLM

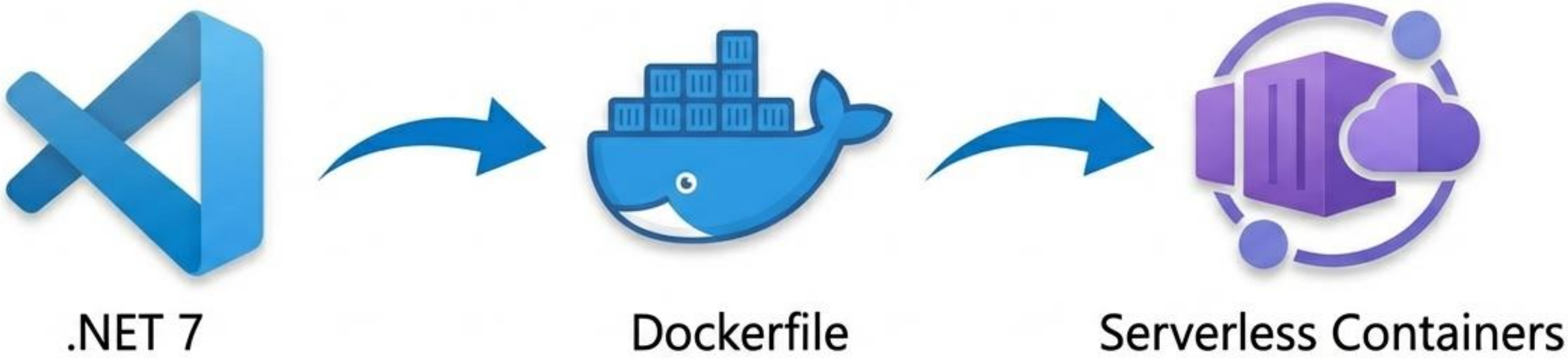
El Ciclo de Vida: The Golden Path



NotebookLM

Caso de Estudio: API .NET + Docker

- **Proyecto**: Minimal API en C#
- **Containerización**: Empaquetado con Docker
- **Infraestructura**: Azure Container Apps
- Sin intervención manual en servidores



Objetivo: Cambiar 'Hello World' a 'Hello Azure DevOps' automáticamente.

NotebookLM

Anatomía del Pipeline (YAML)

```yaml
trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- task: DotNetCoreCLI@2
  displayName: 'Build'
  inputs:
    command: 'build'
```

Evento disparador

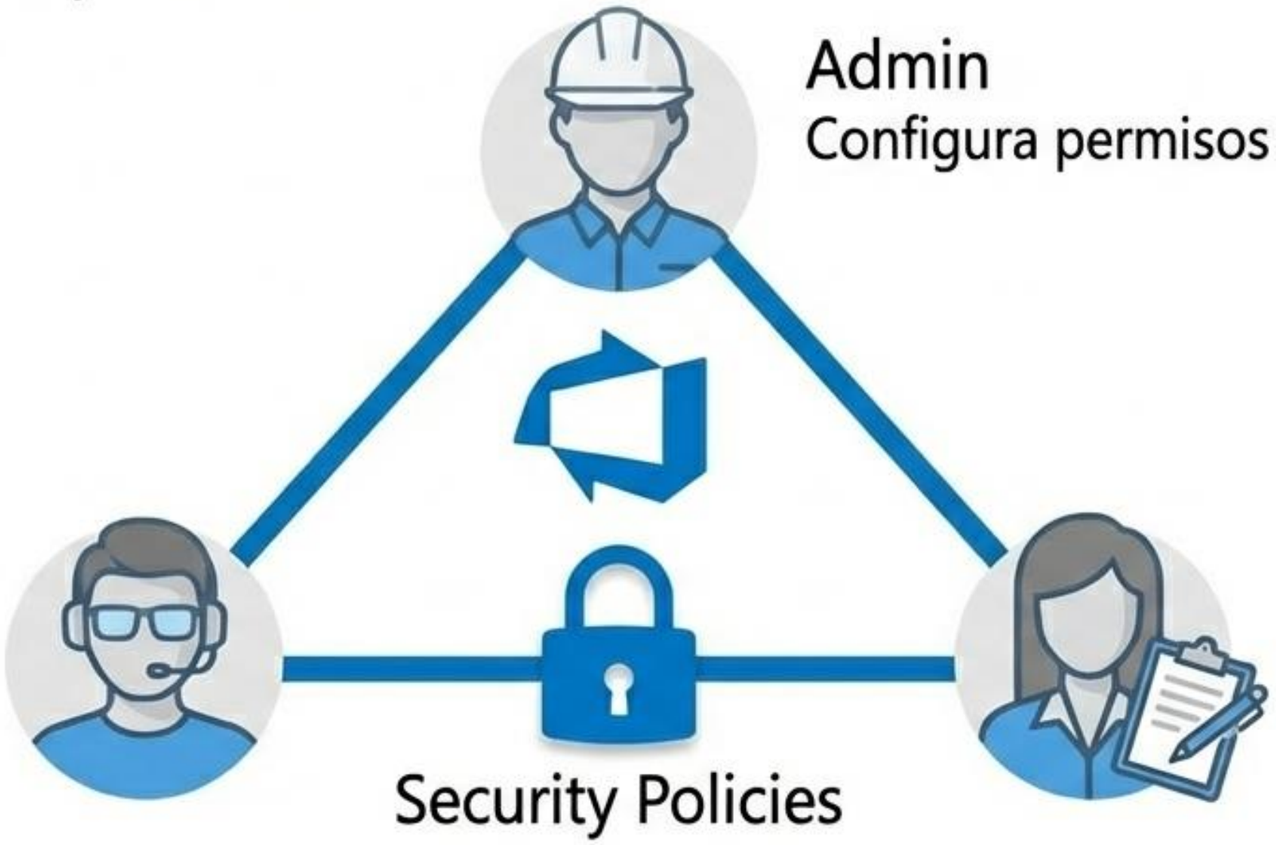
Agente Virtual (Ubuntu)

Paso específico (Compilar)

NotebookLM

Roles y Seguridad

- **Admin:** Gestión de Branch Policies
- **Developer:** Ejecución técnica
- **Seguridad:** Nadie aprueba sus propios cambios



NotebookLM

Buenas Prácticas (Pro-Tips)

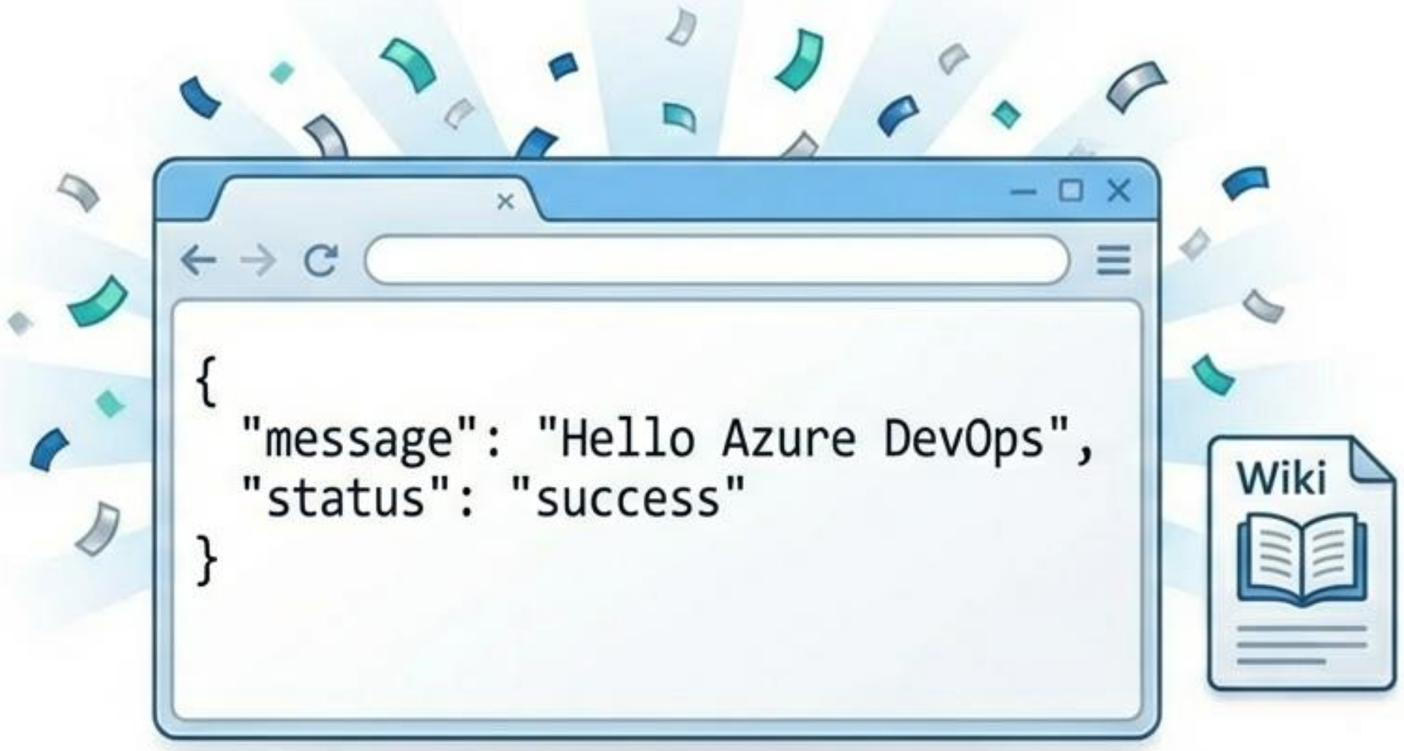
Consejos para un DevOps Profesional

- ✅ Vincula ramas a tareas (ej. user/123-feature)
- ✅ Pull Requests pequeños y atómicos
- ✅ Automatiza todo (Infra como Código)
- ✅ Separa responsabilidades (Build vs Deploy pipelines)
- ✅ NUNCA hagas commit directo a Main

NotebookLM

Resumen y Siguientes Pasos

- **Logro:** API desplegada y segura en Azure
- **Documentación:** Uso de la Wiki del proyecto
- **Monitoreo:** Revisión de métricas en tiempo real
- ¿Qué proceso manual eliminarás hoy?



NotebookLM